

Lab 1: Raw 8-Bit Ethernet Verification

Hardware-in-the-Loop Simulation & Waveform Analysis

Zuñiga, Ivan Andrés

August 25, 2026

Fundación Fulgor

1. System Architecture & Simulation Datapath

Hardware-in-the-Loop Setup:

- **Host Interface:** tap0 (10.0.0.1/24).
- **Target Interface:** tap1 in namespace ns_b (10.0.0.2/24).
- **C++ Testbench (wrapper.cpp):**
Non-blocking TAP descriptor I/O streaming bytes to and from RTL.
- **RTL Core (top.sv):** Synchronous 8-bit bidirectional register datapath.
- **Framing:** Raw L2 frame stream, preamble and FCS handled at physical PHY layer.

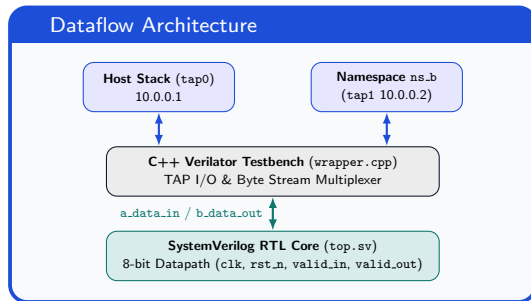


Figure 1: Simulation dataflow connecting Linux TAP interfaces to the SystemVerilog RTL core.

2. Environment Setup & Traffic Generation

Execution Sequence:

1. TAP Network Initialization:

```
sudo ./setup_tap.sh
```

Creates tap0 and isolated namespace ns_b with tap1.

2. Verilator Build & Run:

```
verilator --cc --trace --exe --build
```

```
wrapper.cpp top.sv
```

```
sudo ./obj_dir/emulator
```

3. Packet Capture:

```
sudo tcpdump -i tap0 -w
```

```
lab1_capture.pcap
```

4. Pattern Traffic Injection:

```
ping -c 2 -p cafe0001 -I tap0 10.0.0.2
```

```
> sudo ./obj_dir/emulator
[TAP] Interfaz tap0 vinculada correctamente (host).
[TAP] Interfaz tap1 vinculada correctamente (ns_b).
[EMULADOR LAB 1] Ejecutando simulación...

> sudo ~/.local/bin/tcpdump -i tap0 -w lab
1_capture.pcap
tcpdump: listening on tap0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C4 packets captured
4 packets received by filter
0 packets dropped by kernel

> ping -c 2 -p cafe0001 -I tap0 10.0.0.2
PATTERN: 0xcafe0001
PING 10.0.0.2 (10.0.0.2) from 10.0.0.1 tap0: 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=2.84 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=2.79 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 2.787/2.815/2.843/0.028 ms
```

Figure 2: Terminal execution showing TAP interface binding, Verilator build, and no loss in ICMP traffic.

3. Layer 2 / Layer 3 Wireshark Packet Analysis

Captured ICMP Echo Request Frame (98 Bytes Total):

- **Layer 2 Header (14B):** Dst MAC 02:00:00:00:00:02, Src MAC 02:00:00:00:00:01, EtherType 0x0800 (IPv4).
- **Layer 3 Header (28B):** IPv4 10.0.0.1 → 10.0.0.2 (20B), ICMP Type 8 (Echo Request, Code 0, 8B).
- **Payload (56B):** 16-byte kernel timestamp + 40-byte repeating pattern `ca fe 00 01`.

> Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits)	0000	02 00 00 00 00 02 02 00	00 00 00 01 08 00 45 00
> Ethernet II, Src: 02:00:00:00:00:01 (02:00:00:00:00:01), Dst: 02:00:00:00:00:02 (02:00:00:00:00:02)	0010	00 54 56 60 40 00 40 01	d0 46 0a 00 00 01 0a 00
> Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2	0020	00 02 08 00 aa f7 3a d6	00 01 d8 f6 84 6a 00 00
> Internet Control Message Protocol	0030	00 00 bc d1 0a 00 00 00	00 00 ca fe 00 01 ca fe
	0040	00 01 ca fe 00 01 ca fe	00 01 ca fe 00 01 ca fe
	0050	00 01 ca fe 00 01 ca fe	00 01 ca fe 00 01 ca fe
	0060	00 01	

Figure 3: Wireshark dissection and hex dump of the 98-byte captured ICMP Echo Request frame.

4. SW-to-HW Correlation: Layer 2 Header

Ethernet II Header Stream Alignment (14 Bytes):

- **Dst MAC (Bytes 0–5):** 02 00 00 00 00 02 $\longleftrightarrow$ Clock Cycles 0–5 on a_data_in[7:0].
- **Src MAC (Bytes 6–11):** 02 00 00 00 00 01 $\longleftrightarrow$ Clock Cycles 6–11 on a_data_in[7:0].
- **EtherType (Bytes 12–13):** 08 00 (IPv4) $\longleftrightarrow$ Clock Cycles 12–13 on a_data_in[7:0].

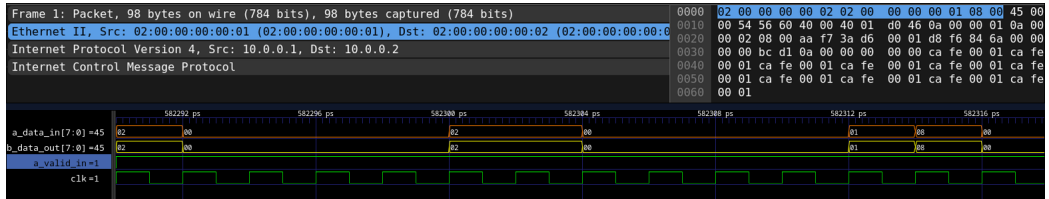


Figure 4: Cycle-by-cycle alignment between Wireshark Layer 2 hex dump and GTKWave signal transitions.

5. SW-to-HW Correlation: Layer 3 IPv4 Header

IPv4 Header Stream Alignment (20 Bytes):

- **Version / IHL / Length (Bytes 14–17):** 45 00 00 54 (IPv4, 20B Header, 84B Total) $\longleftrightarrow$ Clock Cycles 14–17.
- **ID / Flags / TTL / Proto (Bytes 18–23):** 56 60 40 00 40 01 (TTL 64, ICMP Proto 1) $\longleftrightarrow$ Clock Cycles 18–23.
- **Checksum / IP Endpoints (Bytes 24–33):** D0 46 (Checksum), 0A 00 00 01 (Src), 0A 00 00 02 (Dst) $\longleftrightarrow$ Cycles 24–33.

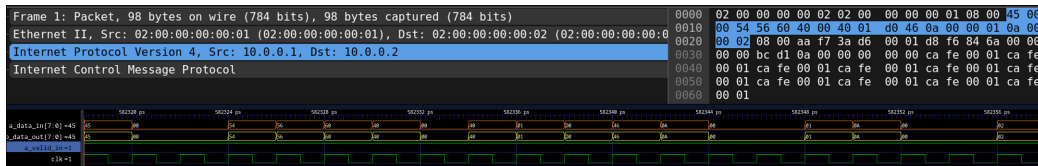


Figure 5: Cycle-by-cycle hardware bus matching for the 20-byte IPv4 packet header stream.

6. SW-to-HW Correlation: ICMP & Payload

ICMP Header & Payload Stream Alignment (64 Bytes):

- **ICMP Header (Bytes 34–41):** 08 00 AA F7 3A D6 00 01 (Type 8, Code 0, Checksum, ID, Seq) $\longleftrightarrow$ Clock Cycles 34–41.
- **Timestamp Data (Bytes 42–57):** D8 F6 84 6A ... BC D1 0A 00 ... $\longleftrightarrow$ Clock Cycles 42–57.
- **Custom Payload Pattern (Bytes 58–97):** Repeating signature CA FE 00 01 $\longleftrightarrow$ Clock Cycles 58+ on a_data_in[7:0].

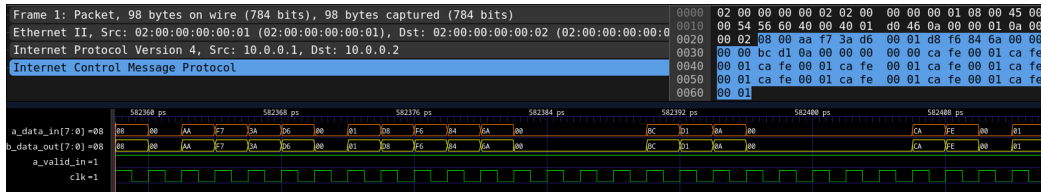


Figure 6: Hardware bus streaming of the ICMP header and custom 0xCAFE0001 repeating payload pattern.

7. Pipeline Latency & Cycle Delay Analysis

RTL Registered Datapath Verification (top.sv):

- **Setup Phase (clk=0):** Input byte a_data_in = 0x02 and a_valid_in = 1'b1 are driven during the posedge.
- **Clock Edge Latch (clk=1):** Output valid b_valid_out and data b_data_out = 0x02 assert on the rising edge.
- **1-Cycle Latency:** wrapper.cpp drives inputs on clk=0 to model the setup window for RTL non-blocking assignments ($\leq$), giving a 1-clock delay ($T_{\text{latency}} = 1 T_{\text{clk}}$) through the D-FF register stage.

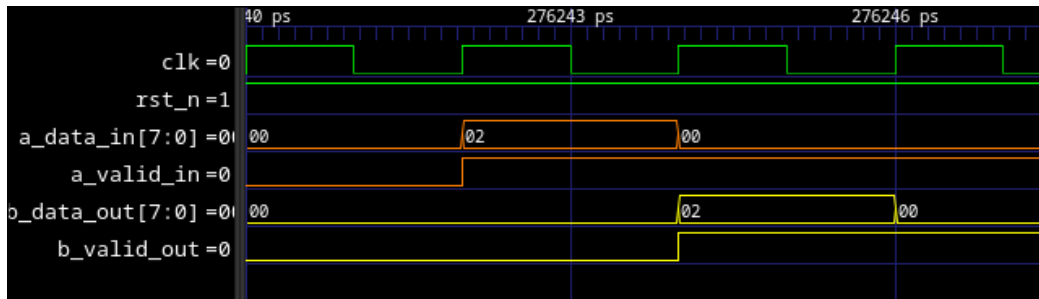


Figure 7: GTKWave cycle zoom displaying the exact 1-clock-cycle delay between input assertion and registered output valid.

Thank You!